



HARVARD
MEDICAL SCHOOL

BLAVATNIK INSTITUTE
BIOMEDICAL INFORMATICS

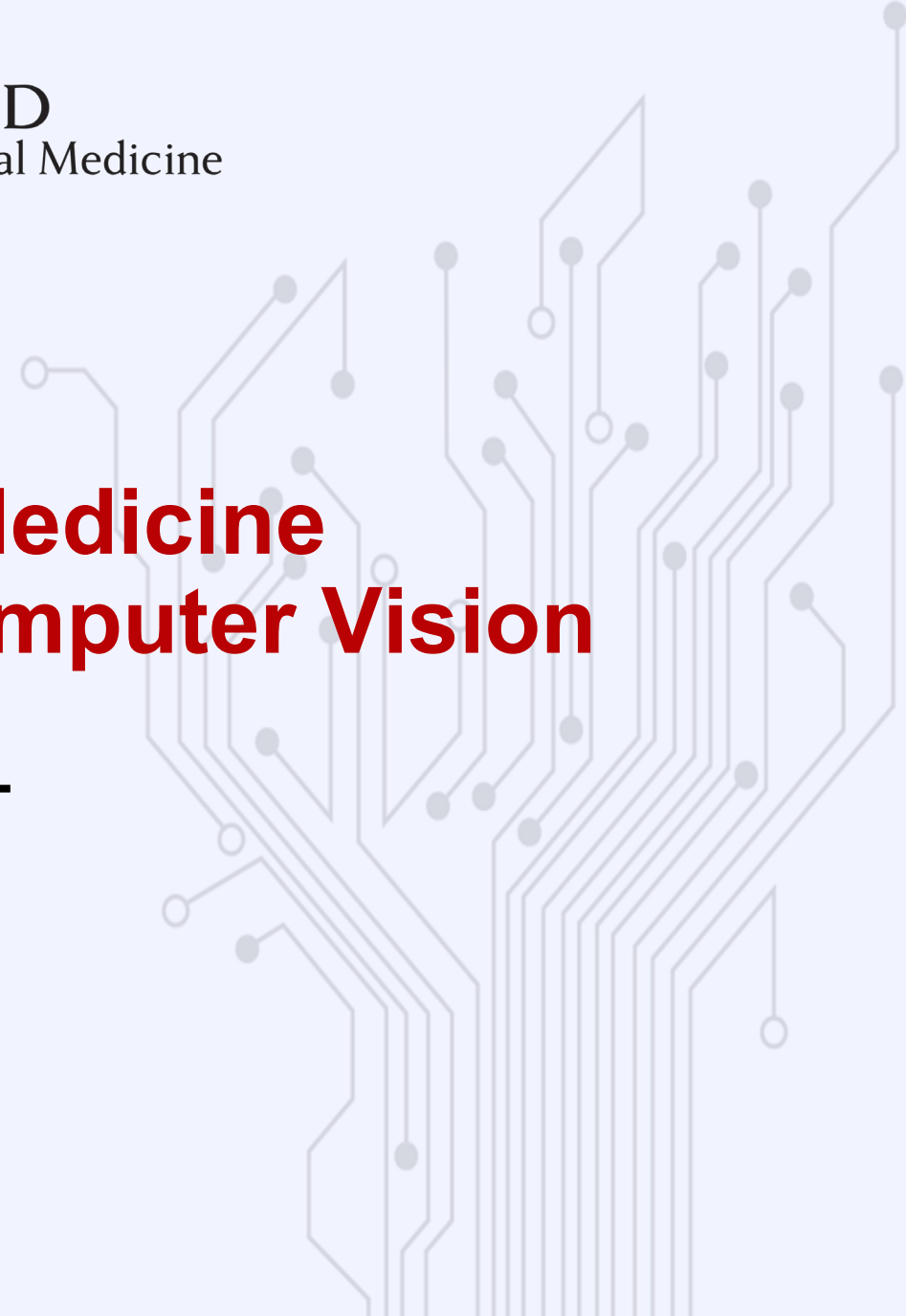


HARVARD
School of Dental Medicine

Artificial Intelligence in Dental Medicine Augmented Diagnostics and Computer Vision

Part 2 07/15/2026

Andreas Werdich PhD



Agenda

Part 1: Introduction, data, and testing

- Computer vision and AI
- Getting data for training models
- Working with clinical data, PHI
- Part 2: Training a vision transformer model
- Evaluating model performance

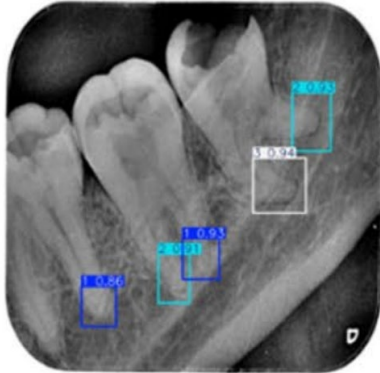
Part 2: Training a detection transformer model

- More “technical” session than part 1
- Goal: provide resources to train your own model
- Getting started with code

The computer vision repository

- DETR: State-of-the-art detection transformer
- Model training process
- Use your own data

Common tasks

Task	Use case		Model output
Image classification	“Is the quality good enough, or do I need to retake it?”		Class label: “Incomplete apical coverage” Deep learning-based assessment of periapical radiographic image quality. Chi et al. Nature Scientific Reports (2026)
Object detection	“Find and classify apical periodontitis”		Bounding boxes with periapical index (PAI) scores Automated assessment of periapical health based on the radiographic index using one-stage object detection algorithms. Saber et al. Nature Scientific Reports (2025)
Segmentation	“Measure periodontal bone loss”		Radiographic bone level (RBL) percentage rate Automatic recognition of teeth and periodontal bone loss measurement in digital radiographs using deep-learning artificial intelligence. Chen et al. J. Dent. Sc. (2023)

Tooth detection and disease classification

Task	Use case	Model output
Object detection	“Identify the teeth shown in the radiograph”	 Bounding boxes with tooth numbers A novel deep learning-based perspective for tooth numbering and caries detection Ayhan et al. Clinical Oral Investigations (2024)
• “Finding things ...” important task in radiography analysis • Subject of intensive research • Foundation for AI models in dentistry (treatment planning, decision support, surgical planning, ...)		

No-code, is it an option?

Claude Opus 4.8



You are a powerful clinical AI system. You are an expert at analyzing dental radiographs and diagnosing oral diseases. Given the attached image, please complete the following tasks:

- (1) Describe the image*
- (2) Provide the number of teeth you see in this image*
- (3) Return the bounding box coordinates of the teeth identified in task (2)*
- (4) Return also a list of tooth numbers for the teeth identified in task (2)*

No-code alternatives

Claude Opus 4.8



(1) Image Description

This is a periapical radiograph capturing both a maxillary (upper) and mandibular (lower) posterior segment, most likely from the molar/premolar region.

Key observations:

Maxillary (upper) teeth: Several posterior teeth are visible. Two of the mid-upper teeth show radiopaque occlusal restorations (bright fillings on the biting surface). The most distal upper tooth (far left in the image) appears tilted/mesially inclined, which can occur with drifting or a partially erupted/impacted tooth. Roots and the surrounding trabecular bone are visible superiorly.

Mandibular (lower) teeth: The lower portion shows tooth roots with reasonably distinct root canals and normal-appearing periodontal ligament spaces. Crowns are partly cut off at the image edge.

Overall: The interdental bone levels look grossly maintained in the visible areas, though a definitive read on caries, periapical pathology, or bone loss can't be made from this single low-resolution frame.

No-code alternatives

Claude Opus 4.8



(2) Number of Teeth

I count approximately 6 teeth (roughly 4 in the upper arch and 2 with clearly visible roots in the lower arch). A couple are cut off at the image borders, so the true count depends on how partial teeth are scored.

(3) Bounding Box Coordinates

I want to be honest with you here: I can't produce reliable pixel-level bounding boxes by visual inspection alone. Accurate boxes require an actual object-detection model run against the image file, and any coordinates I "read off" would be rough guesses that could be misleading in a clinical or annotation workflow.

Please estimate the bounding box coordinates and tooth numbers.

(4) ADA (Universal) Tooth Numbers

Based on the appearance (adjacent maxillary and mandibular posterior teeth with a tilted distal upper tooth), this is consistent with a molar region view, but I can't responsibly assign specific numbers like "1, 2, 3, 30, 31" without the laterality information.

The limitations of LLMs

ChatGPT, Gemini, Claude

- Lack of domain knowledge (could be overcome by fine-tuning)
- Hallucinations, making stuff up
- Be confident about false results
- No confidence scores

Other problems

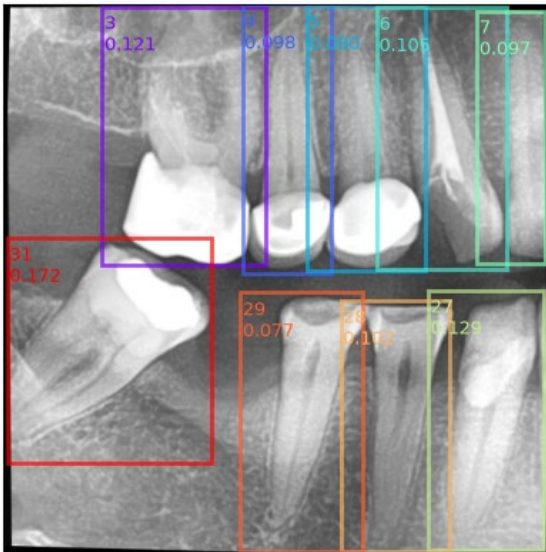
- Resource intensive
- Slow (1-5 minutes per response)
- Risk of PHI exposure



Computer vision models outperform multi-modal LLMs and can run locally on your own hardware.

What you get from a model trained for object detection

Trained RT-DETR model on dental radiographs

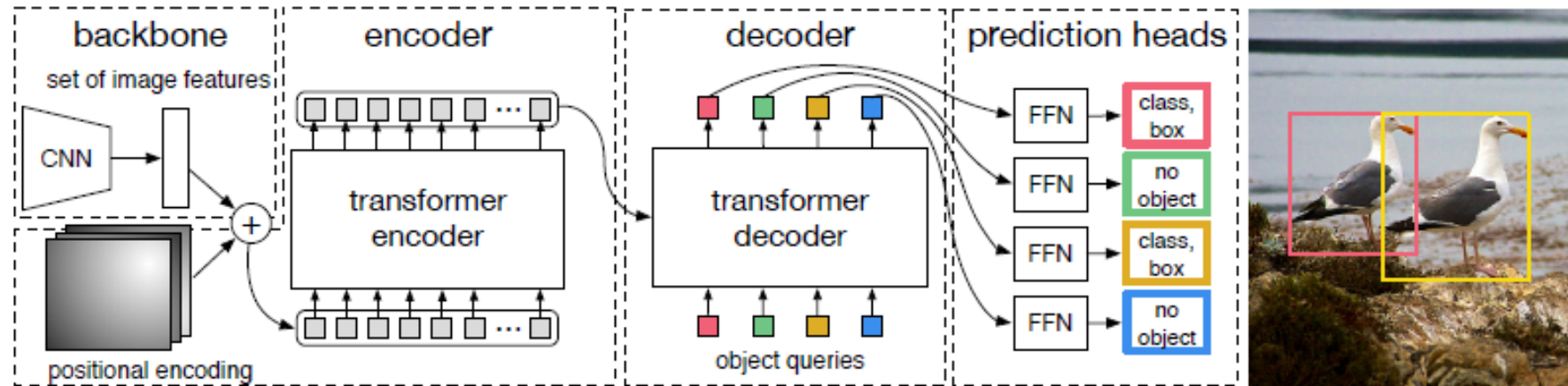


- Pixel-level bounding box coordinates
- Class labels (tooth positions)
- Confidence score for each prediction
- Expected performance of the model across categories

Computer vision models use proven high-performance architectures for pattern recognition

DETR Model

Vision transformer model for tooth position detection



DETR: Detection Transformer

- Architecture developed by Meta AI (2020)
- Most powerful architecture for object detection
- Outperforms all previous architectures
- Combines CNN and Transformer
- Currently beats all others (YOLO, R-CNN)

Main components

1. CNN backbone for feature extraction (ResNet)
2. Transformer for sequence prediction
3. Feed-forward neural network for final prediction

<https://arxiv.org/abs/2005.12872>

GitHub - facebookresearch/detr: End-to-End Object Detection with Transformers

YouTube: <https://www.youtube.com/watch?v=r3IDNWYDGF4&t=541s>

Why DETR is a good choice

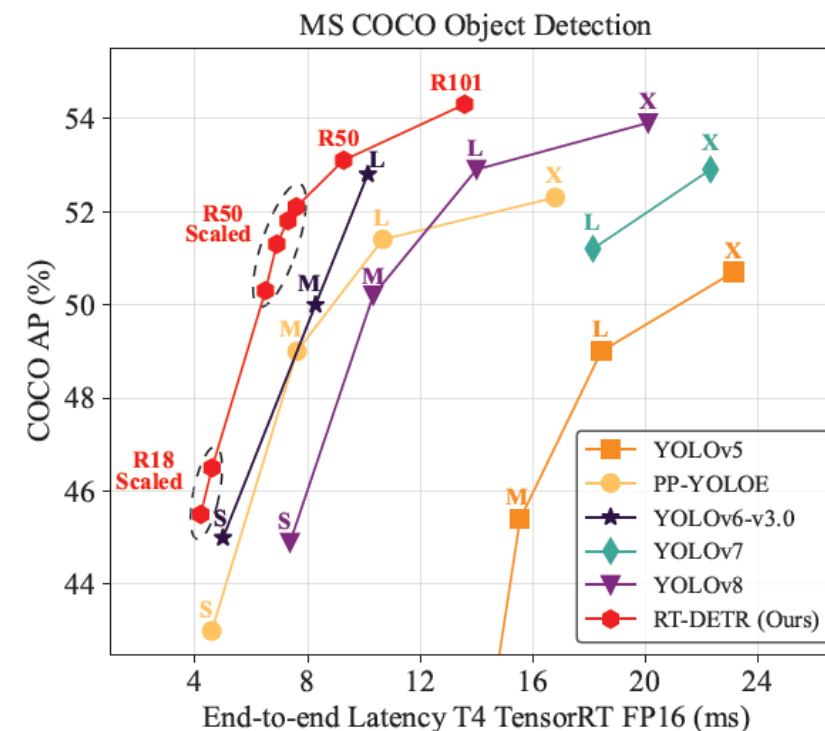
DETRs Beat YOLOs on Real-time Object Detection

Yian Zhao^{1,2†} Wenyu Lv^{1†‡} Shangliang Xu¹ Jinman Wei¹ Guanzhong Wang¹
Qingqing Dang¹ Yi Liu¹ Jie Chen^{2✉}

¹Baidu Inc, Beijing, China ²School of Electronic and Computer Engineering, Peking University, Shenzhen, China

zhaoyian@stu.pku.edu.cn lvwenyu01@baidu.com jiechen2019@pku.edu.cn

- Competitive: outperforms existing architectures
- Extensively used in computer vision research
- Matches important clinical task:
 bounding box with label and confidence score
- Established end-to-end training pipeline
- Use with any labeled data set (caries, implants, restorations)



Do I need to be an expert in Python?

- It's important to know the basics
 - Variables, basic types (numbers, strings, lists, ...)
 - Data frames
 - Functions and classes
 - Loading data
- Resources
 - YouTube
 - Commercial sites (Coursera, Udemy, ...)

You can run the code in the repository, but
it's more fun when you can read it.

Videos :



Python Full Course for free

YouTube · Bro Code ·
Aug 20, 2024

In this video I'm going to teach you everything you need to know to start coding with **Python**.



Python Full Course for Beginners

YouTube · Programming with Mosh ·
Feb 12, 2025

Learn **Python** from scratch with this comprehensive **course**. Master basics, advanced concepts, and build real-world applications.



Python Full Course for Beginners (13 Hours) – From Zero to ...

YouTube · Data with Baraa ·
Feb 24, 2026

Learn **Python** with a unique **course** that explains how **Python** actually works behind the scenes, from the basics to advanced topics like data structures and functions.



I Tried 50 Python Courses. Here Are Top 5.

YouTube · Pooja Dutt ·
May 8, 2025

The code repository

<https://github.com/ccb-hms/computervision>

- Handwritten code (no AI slob!)
- Working end-to-end pipelines
 - Image classification
 - Object detection
 - Instance segmentation
- Detailed README file with install instructions
- Open-source
 - Data sets
 - Libraries (Hugging Face Transformers)
 - PyTorch deep learning framework

📖 README 📄 CC-BY-4.0 license

Getting started with this repository

- [Install on your local machine](#)
- [Install on the HMS O2 cluster](#)
- [Docker setup reference](#)
- [Environment variables](#)
- [Data sets](#)
- [Classification pipeline](#)
- [Object detection pipeline](#)
- [Instance segmentation pipeline](#)
- [Repository architecture \(\[src/computervision\]\(#\) \)](#)
- [Testing and CI](#)
- [Label Studio](#)

Prerequisites to run the code

Hardware

- A modern computer (laptop, desktop)
- A GPU for training: MacBook with M-series chip, Windows/Linux PC with discrete graphics card
- VRAM: ~ 6 GB is enough for training the vision transformer model

<https://github.com/git-guides/install-git>

Software

- Operating system (macOS, Windows, Linux)
- Git – version control to download the repository
- Python environment

Editor (IDE, e.g. VS Code, PyCharm, SublimeText)

UV Python package manager

OR: Docker desktop (Windows, Linux)

```
~ % git --version  
~ % brew install git
```

<https://docs.docker.com/>



Run without Docker

Using the UV Python package manager

```
git clone git@github.com:ccb-hms/computervision.git
cd computervision
uv sync --frozen
```

```
cp env .env
```

```
# Location for the data if not using Docker
DATA_DIR=/home/andreas/data/cv_data
CL_URL=https://dsets.s3.us-east-1.amazonaws
DT_URL=https://dsets.s3.amazonaws.com/dente
HF_TOKEN=hf_token
```

```
uv run jupyter lab
```

Run with Docker

Use Docker if you don't want to deal with Python dependencies

```
git clone git@github.com:ccb-hms/computervision.git
cd computervision
cp env .env          # then fill in real values, see docs
./compose-up.sh
```

```
[I 2026-07-14 23:53:08.646 ServerApp] Serving notebooks from local directory:
[I 2026-07-14 23:53:08.646 ServerApp] Jupyter Server 2.20.0 is running at:
[I 2026-07-14 23:53:08.646 ServerApp] http://0.0.0.0:8888/lab
[I 2026-07-14 23:53:08.646 ServerApp] http://127.0.0.1:8888/lab
[I 2026-07-14 23:53:08.646 ServerApp] Use Control-C to stop this server and s
[W 2026-07-14 23:53:08.652 ServerApp] No web browser found: Error('could not
[I 2026-07-14 23:53:08.665 ServerApp] Skipped non-installed server(s): basedp
ypscript-langserver, jedi-language-server, julia-language-server, pyrefly, py
age-server, texlab, typescript-language-server, unified-language-server, vsco
eserver-bin, yamll-language-server
```

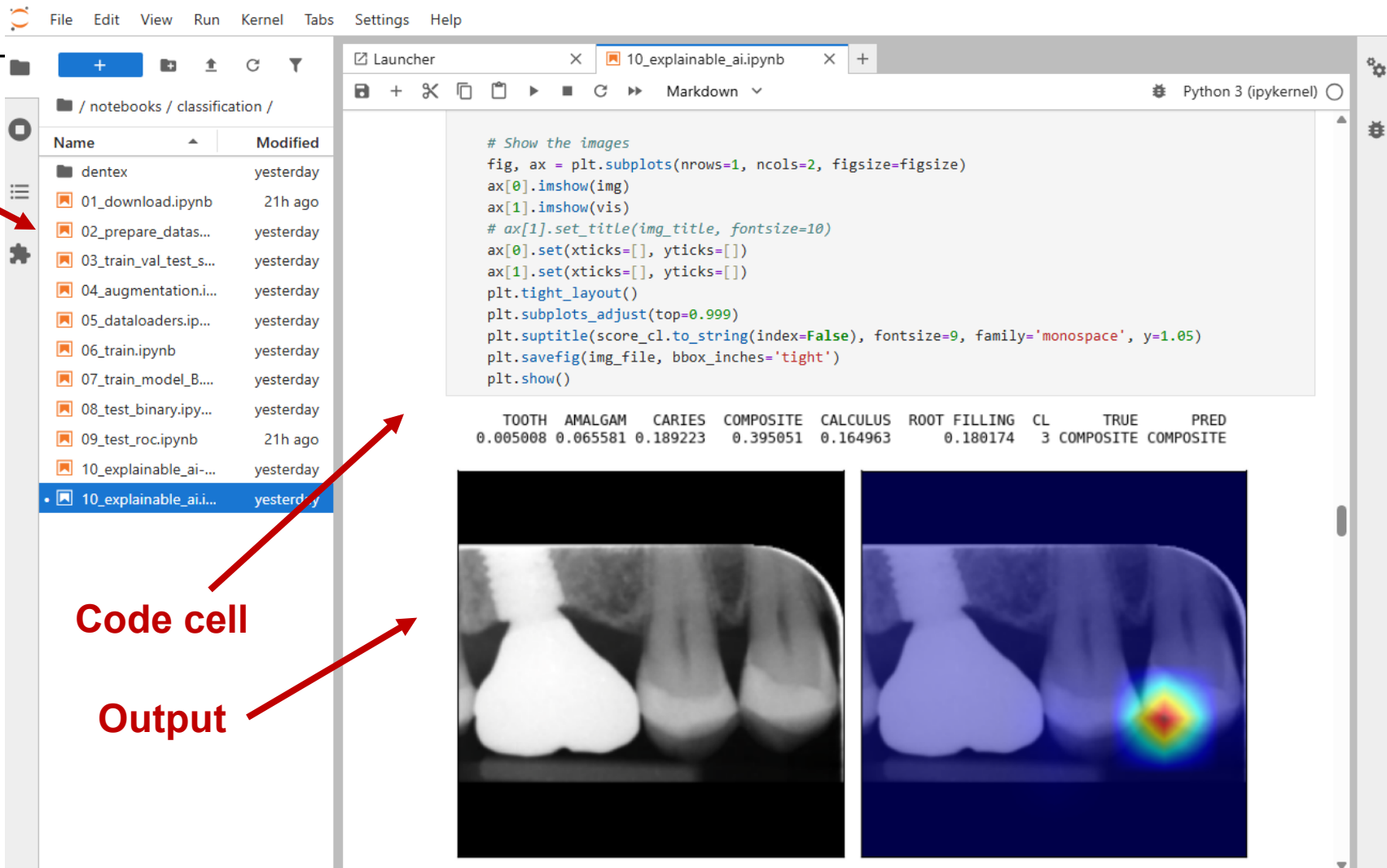
Click on the link to start a browser and see the GUI

The Jupyter Lab GUI

File explorer

Jupyter notebook

- Document
- Combines
 - File explorer
 - Executable code
 - Output
 - Text
 - Graphics
- Exploratory analysis



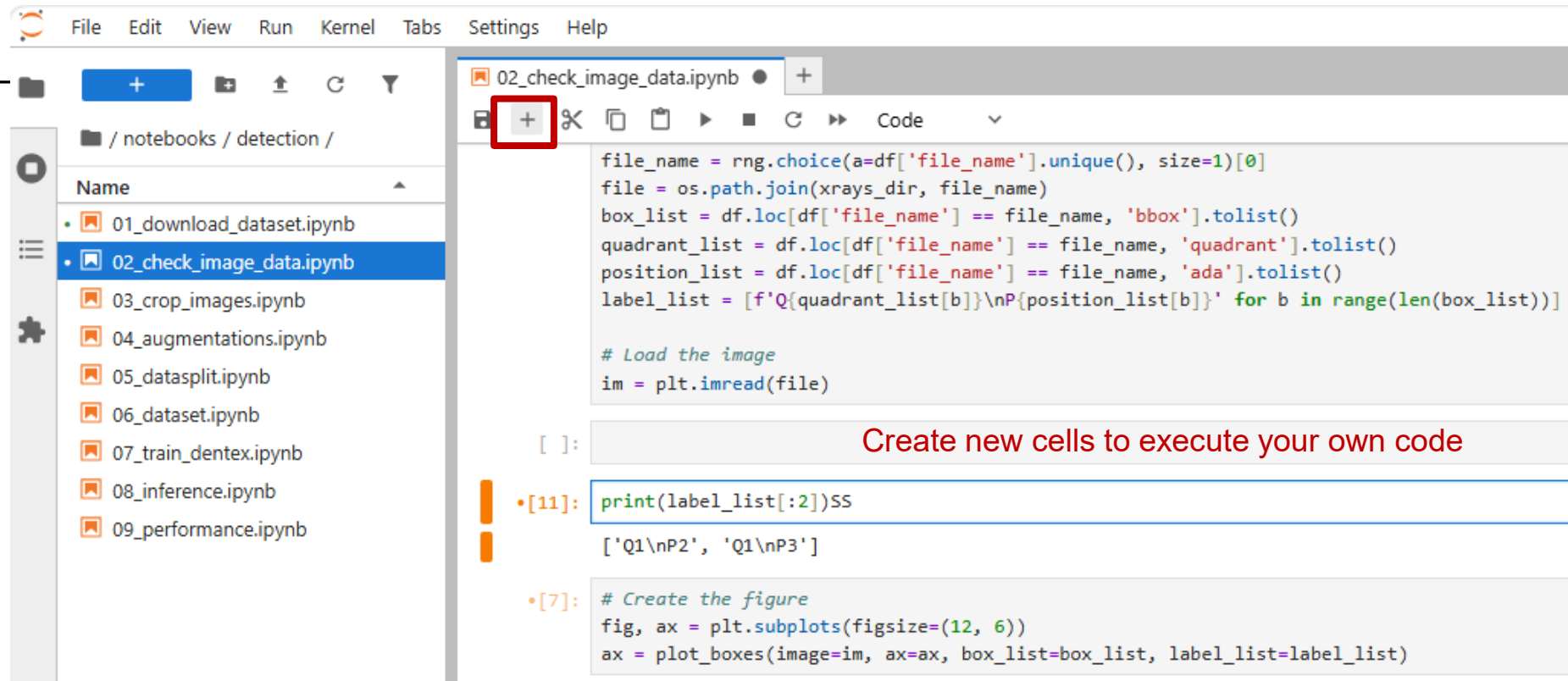
The screenshot displays the Jupyter Lab interface. On the left, a file explorer shows a directory structure with files like '01_download.ipynb' through '10_explainable_ai...'. The main area contains a code cell with Python code for plotting and a table of classification results. Below the table, two side-by-side images are shown: a grayscale dental X-ray and a corresponding heatmap visualization.

```
# Show the images
fig, ax = plt.subplots(nrows=1, ncols=2, figsize=figsize)
ax[0].imshow(img)
ax[1].imshow(vis)
# ax[1].set_title(img_title, fontsize=10)
ax[0].set(xticks=[], yticks=[])
ax[1].set(xticks=[], yticks=[])
plt.tight_layout()
plt.subplots_adjust(top=0.999)
plt.suptitle(score_cl.to_string(index=False), fontsize=9, family='monospace', y=1.05)
plt.savefig(img_file, bbox_inches='tight')
plt.show()
```

TOOTH	AMALGAM	CARIES	COMPOSITE	CALCULUS	ROOT FILLING	CL	TRUE	PRED
0.005008	0.065581	0.189223	0.395051	0.164963	0.180174	3	COMPOSITE	COMPOSITE

How to work with the notebooks

Some suggestions



- Run notebooks in sequence
- Cell-by-cell
- Create new cells to separate steps
- Create new notebooks to run with your own data

What's in the repository

Notebooks to explain the code

- Numbered Jupyter notebooks
- Complete computer vision workflow
- To be executed in sequence
- Links included to download data
- Custom (imported) functions

Load images

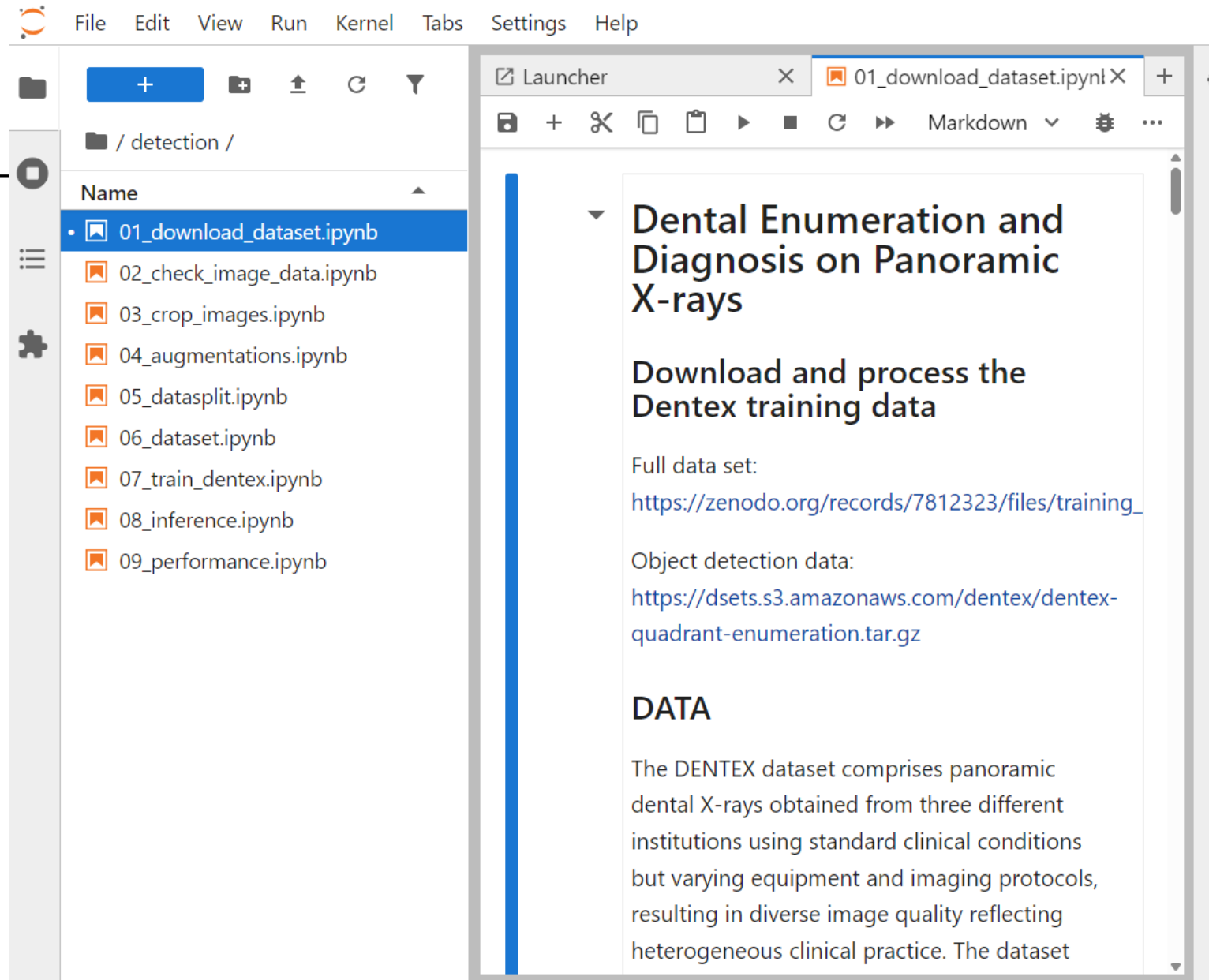
Display images/bounding boxes

Convert bounding box coordinates

Idea:

Understand the workflow

Run it with your own data



The screenshot displays the JupyterLab interface. On the left, the file browser shows a directory structure with a folder named 'detection' containing several Jupyter notebooks (ipynb files): '01_download_dataset.ipynb', '02_check_image_data.ipynb', '03_crop_images.ipynb', '04_augmentations.ipynb', '05_dataloader.ipynb', '06_dataset.ipynb', '07_train_dentex.ipynb', '08_inference.ipynb', and '09_performance.ipynb'. The notebook '01_download_dataset.ipynb' is selected. On the right, the notebook '01_download_dataset.ipynb' is open, showing a title 'Dental Enumeration and Diagnosis on Panoramic X-rays', a subtitle 'Download and process the Dentex training data', and two links: 'Full data set: https://zenodo.org/records/7812323/files/training_' and 'Object detection data: <https://dsets.s3.amazonaws.com/dentex/dentex-quadrant-enumeration.tar.gz>'. Below the links, there is a section titled 'DATA' with a paragraph describing the DENTEX dataset.

Dental Enumeration and Diagnosis on Panoramic X-rays

Download and process the Dentex training data

Full data set:
https://zenodo.org/records/7812323/files/training_

Object detection data:
<https://dsets.s3.amazonaws.com/dentex/dentex-quadrant-enumeration.tar.gz>

DATA

The DENTEX dataset comprises panoramic dental X-rays obtained from three different institutions using standard clinical conditions but varying equipment and imaging protocols, resulting in diverse image quality reflecting heterogeneous clinical practice. The dataset

How do we get the images and bounding boxes into the model?

PyTorch Datasets

- Have the data and annotations on your drive (Notebook 01)
- Link image names and annotations (bounding boxes, labels)
- Create a Dataset from the data frame (Notebook 06)
 - Decouple storage/image processing from model training loop
 - Tells the training function how to load individual samples
- Use the Dataset as an argument for the Trainer method (Notebook 07)

```
trainer = Trainer(model=model,
                  args=training_arguments,
                  train_dataset=train_dataset,
                  eval_dataset=val_dataset,
                  processing_class=processor,
                  data_collator=collate_fn,
                  compute_metrics=eval_compute_metrics_fn)

trainer.train()
```

	file_name	bbox	pos
0	train_0_01.png	[666, 102, 103, 376]	1
1	train_0_01.png	[593, 107, 85, 377]	2
2	train_0_01.png	[531, 69, 85, 368]	3
3	train_0_01.png	[457, 31, 115, 403]	4
4	train_0_01.png	[369, 10, 100, 406]	5
5	train_0_01.png	[228, 54, 150, 347]	6
6	train_0_01.png	[90, 37, 164, 344]	7
7	train_0_01.png	[10, 60, 153, 286]	8
8	train_0_02.png	[10, 55, 100, 376]	1
9	train_0_02.png	[98, 66, 86, 350]	2

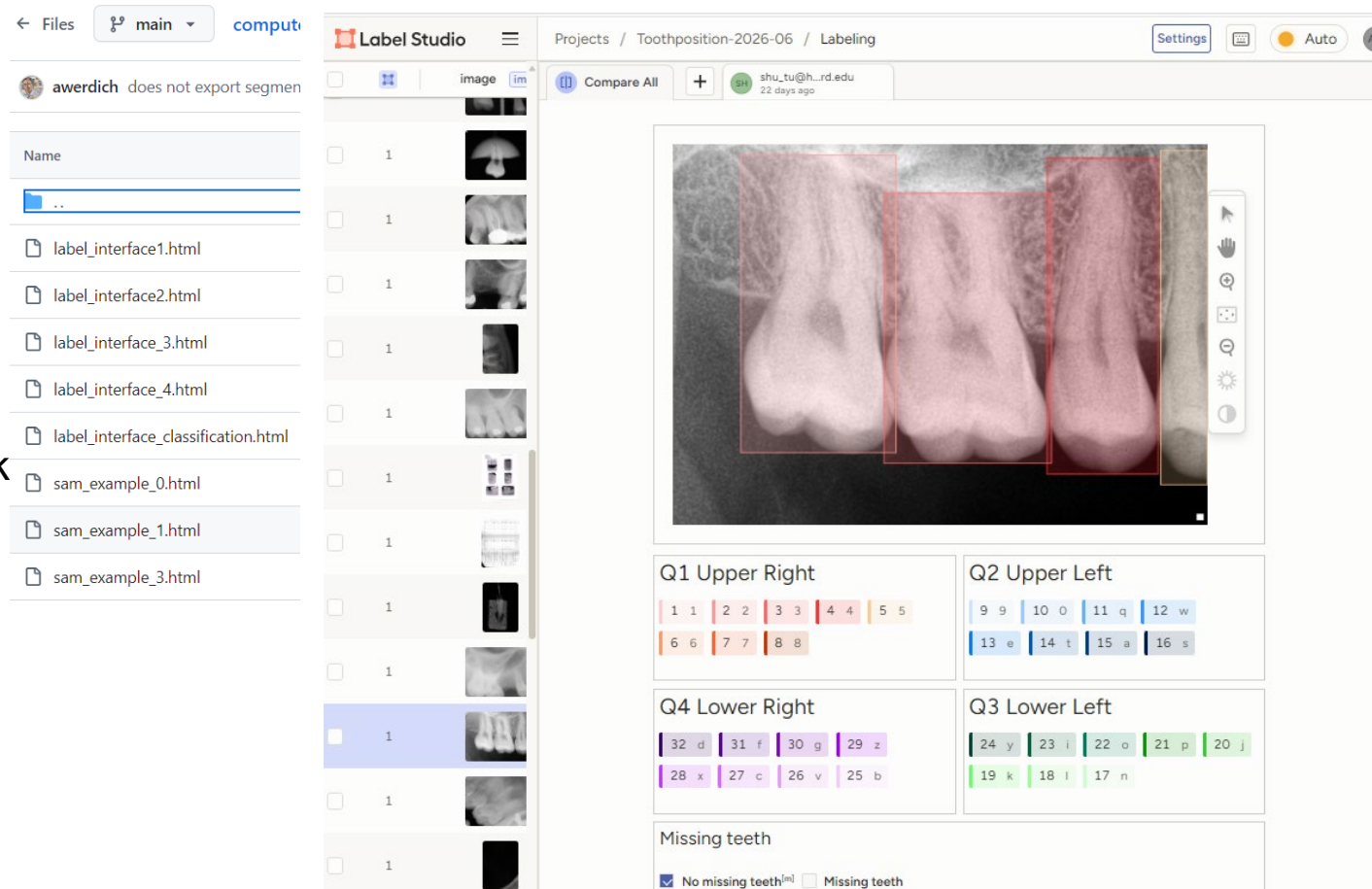
Create a new mapping table to run the code
with your own data.

Label studio for object detection

Manual labeling



- Open-source labeling tool
- Based on Python-Django
- Fully customizable label interface
- Can be run as a server in a secure network
- Access with VPN, 2FA
- Great for collaborative label efforts
- Simultaneous annotation by multiple users
- ML backend for automated pre-annotation



Example label interfaces are included in the repository

<https://labelstud.io/>

Training workflow

Training and testing a detection transformer model



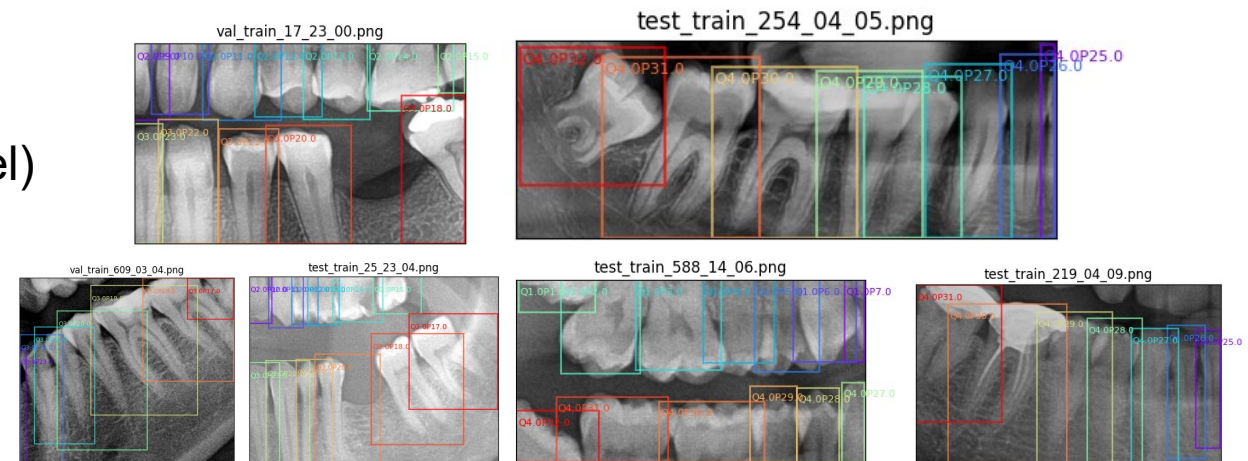
1. Download the data set
2. Prepare the data for training
3. Split into train/val/test sets
4. Configure data augmentation
5. Set up data loaders (to get data into the model)
6. Train the model
7. Test the performance

Image data (2023 Dentex Challenge)

634 panoramic x-rays

Cropped into 5,000 smaller images

20,000 bounding boxes



Data augmentation

Improve the accuracy of your model by creating artificial data



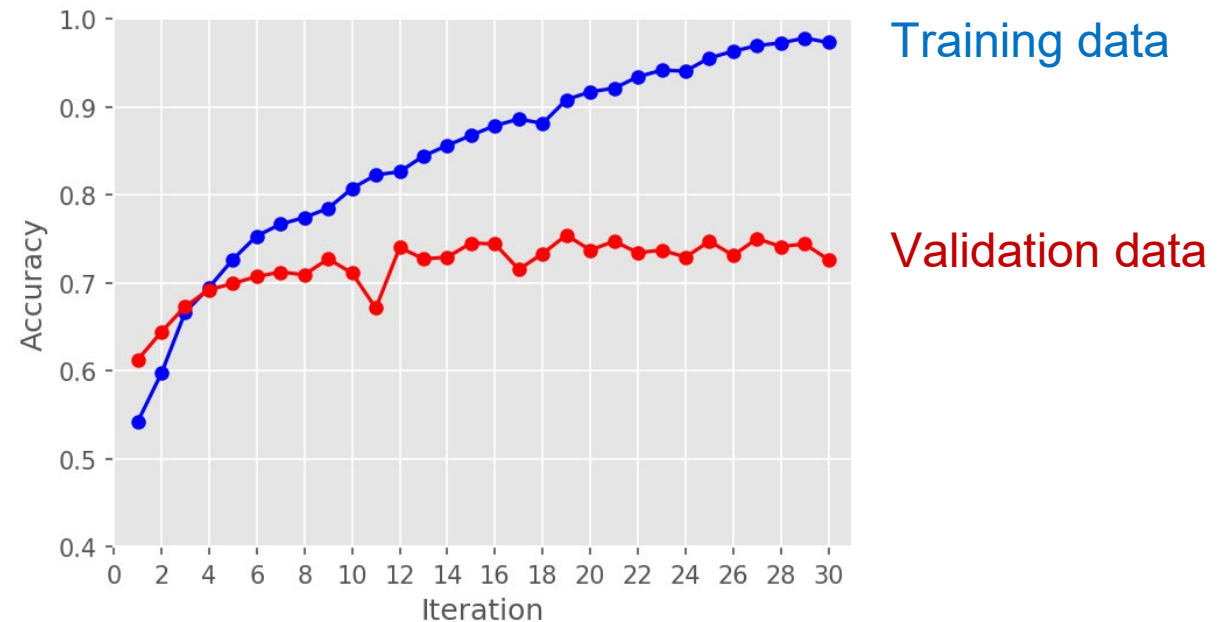
Binary cross-entropy

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Training: Optimize the parameters so that the loss is minimal

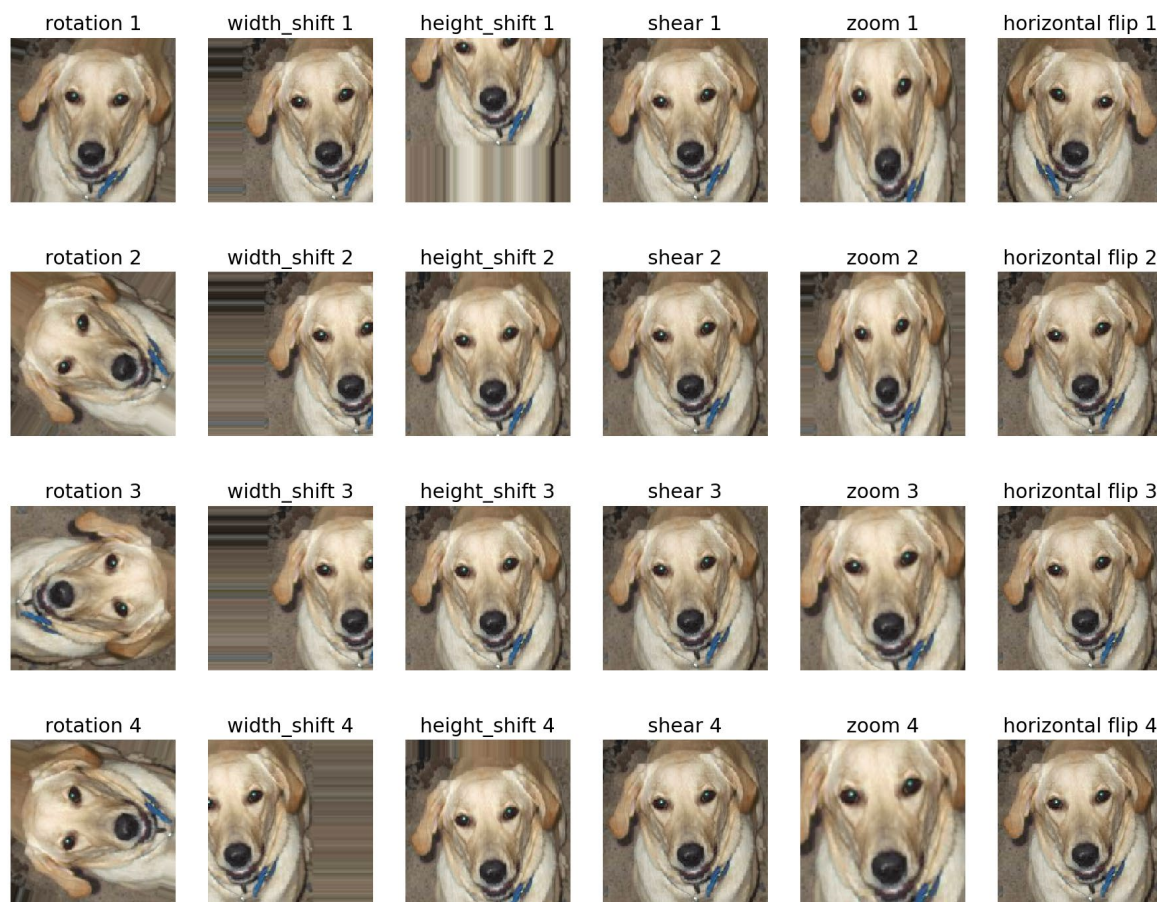
Overfitting

- The model learns the training data perfectly
- Achieves 100% accuracy on training data
- The model stops learning
- Or gets worse
- Does not generalize well to new images

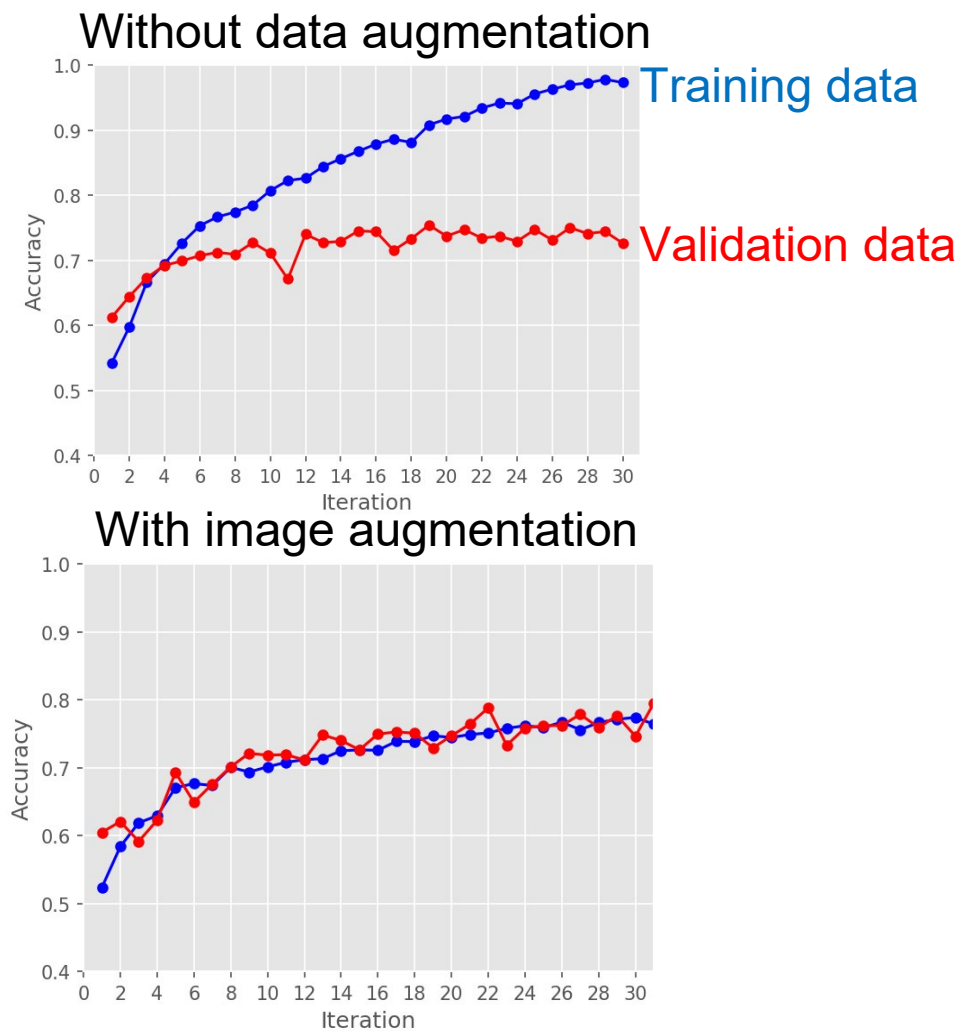


Data augmentation

Improve the accuracy of your model by creating artificial data

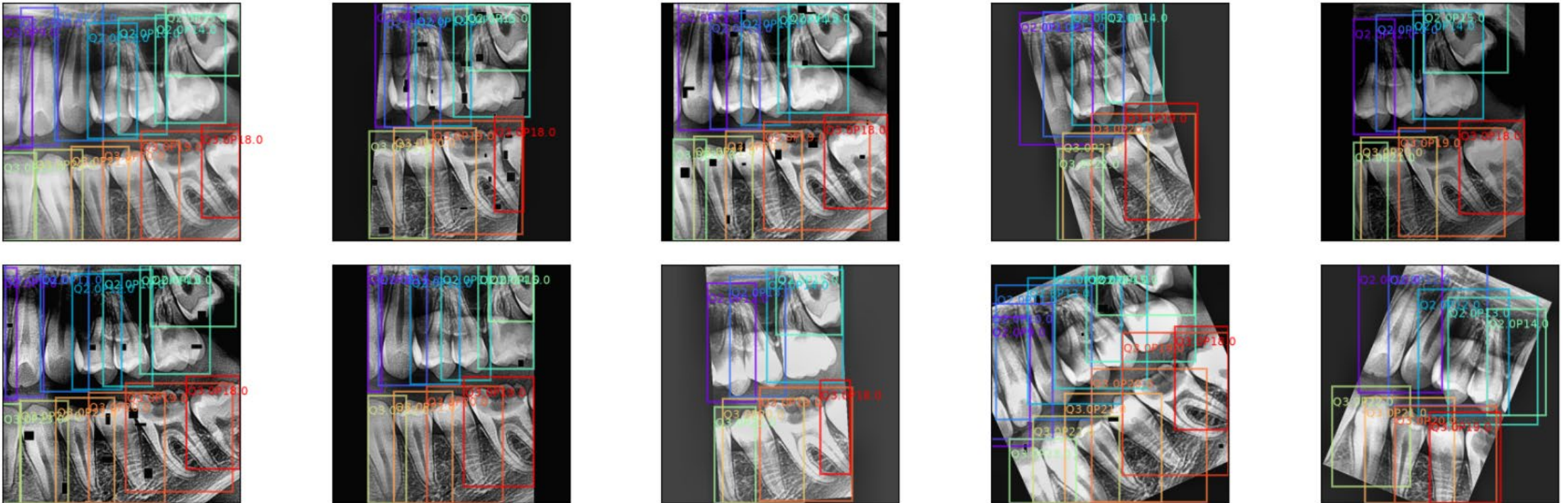


It's a “cheap” way to increase the amount of data for training



Augmentation for object detection

Improve the accuracy of your model by creating artificial data



- Albumentations: open-source library for image augmentation
- Transforms bounding boxes in sync with pixels
- Demonstration in Notebook 04
- Examples in `src/computervision/transformations.py`

<https://albumentations.ai>

The model training (notebook 07)

- Load the data frame with file names and annotations (bounding boxes, class labels)
- Load a pre-trained model and weights from Hugging Face (“PekingU/rtdetr_v2_r101vd”)
- Sets training parameters (e.g., learning rate, augmentations, .. to be optimized)
- Run the training loop
- Report performance metrics

```
trainer = Trainer(model=model,
                  args=training_arguments,
                  train_dataset=train_dataset,
                  eval_dataset=val_dataset,
                  processing_class=processor,
                  data_collator=collate_fn,
                  compute_metrics=eval_compute_metrics_fn)

trainer.train()
```

How good is the trained model?



[1196/1196 43:05, Epoch

Epoch	Training Loss	Validation Loss	Map	Map 50	Map 75	Map Small
1	32.759744	11.949668	0.442700	0.786500	0.434300	-1.000000
2	21.812738	9.919222	0.579500	0.906700	0.646400	-1.000000

Performance metrics for binary classification

Depend on predictions to be “correct” or “incorrect”

- Accuracy: $ACC = \frac{TP+TN}{P+N}$
- Recall: $TPR = \frac{TP}{P} = \frac{TP}{TP+FN}$ (sensitivity)
- Precision: $PPV = \frac{TP}{PP} = \frac{TP}{TP+FP}$
- Specificity: $TNR = \frac{TN}{N}$ (specificity)
- False positive rate: $FPR = \frac{FP}{N}$ (1 – specificity)
- AUC score (area under the ROC curve)
- F1 score: $F1 = 2 \frac{Precision \times recall}{precision + recall}$ (harmonic mean)
- AUC score (area under the ROC curve)

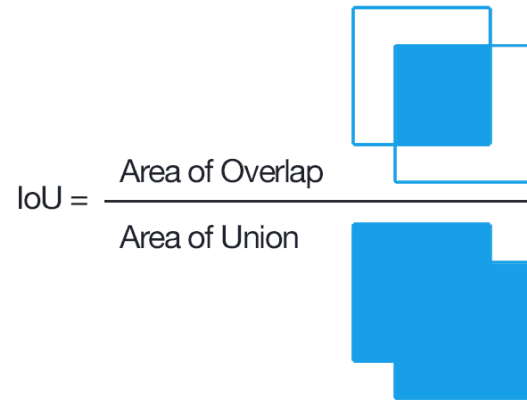
What is “correct” or “incorrect” for an object detection model?

https://en.wikipedia.org/wiki/Precision_and_recall

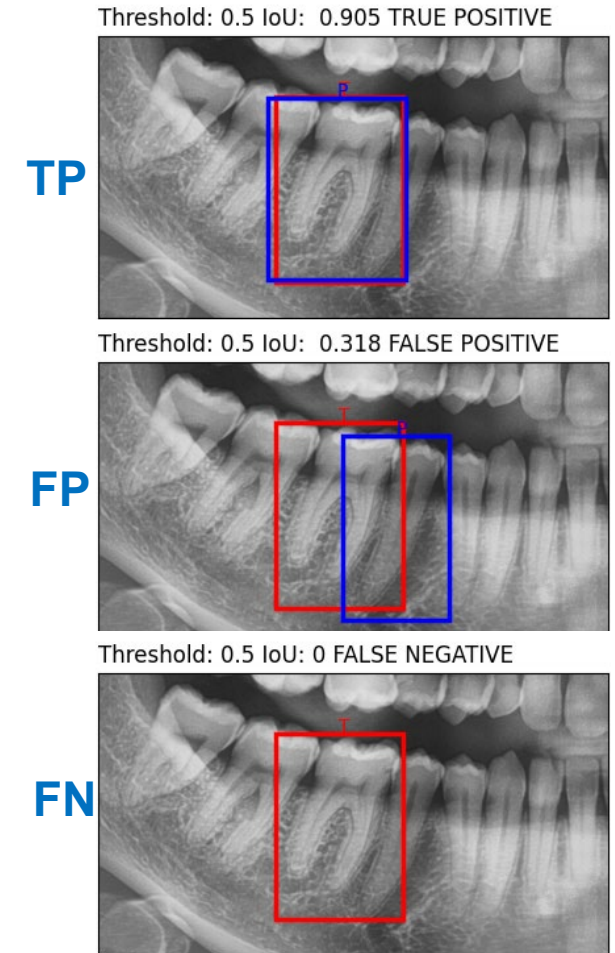
Performance metric for object detection

A correct prediction is

1. A detected bounding box with class label that matches a ground-truth object
2. A detected bbox that overlaps sufficiently with the object's ground-truth bbox

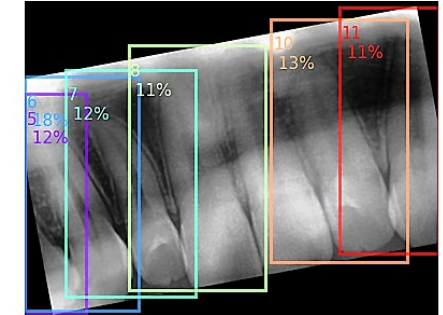
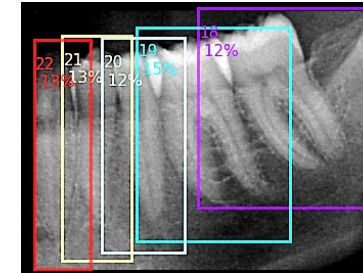
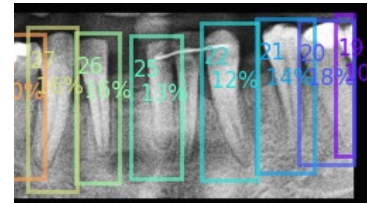
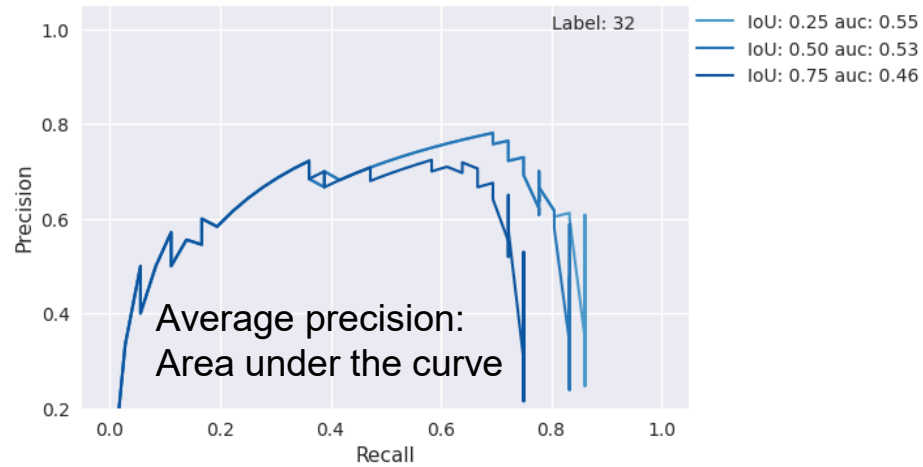


- Recall: $\text{TPR} = \frac{TP}{P} = \frac{TP}{TP+FN}$
- Precision: $\text{PPV} = \frac{TP}{PP} = \frac{TP}{TP+FP}$



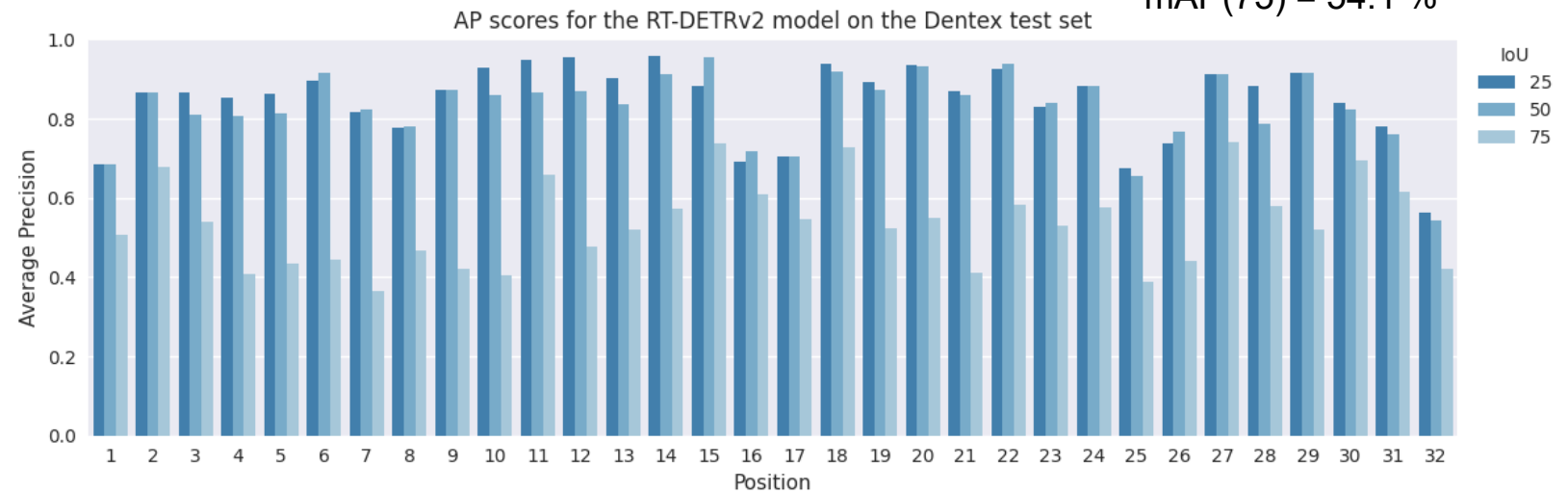
<https://learnopencv.com/mean-average-precision-map-object-detection-model-evaluation-metric/>

Performance of the trained RT-DETR model



mAP(25) = 84.1 %
mAP(50) = 82.8 %
mAP(75) = 54.1 %

AP measurement (Notebook 08)



Contributions welcome

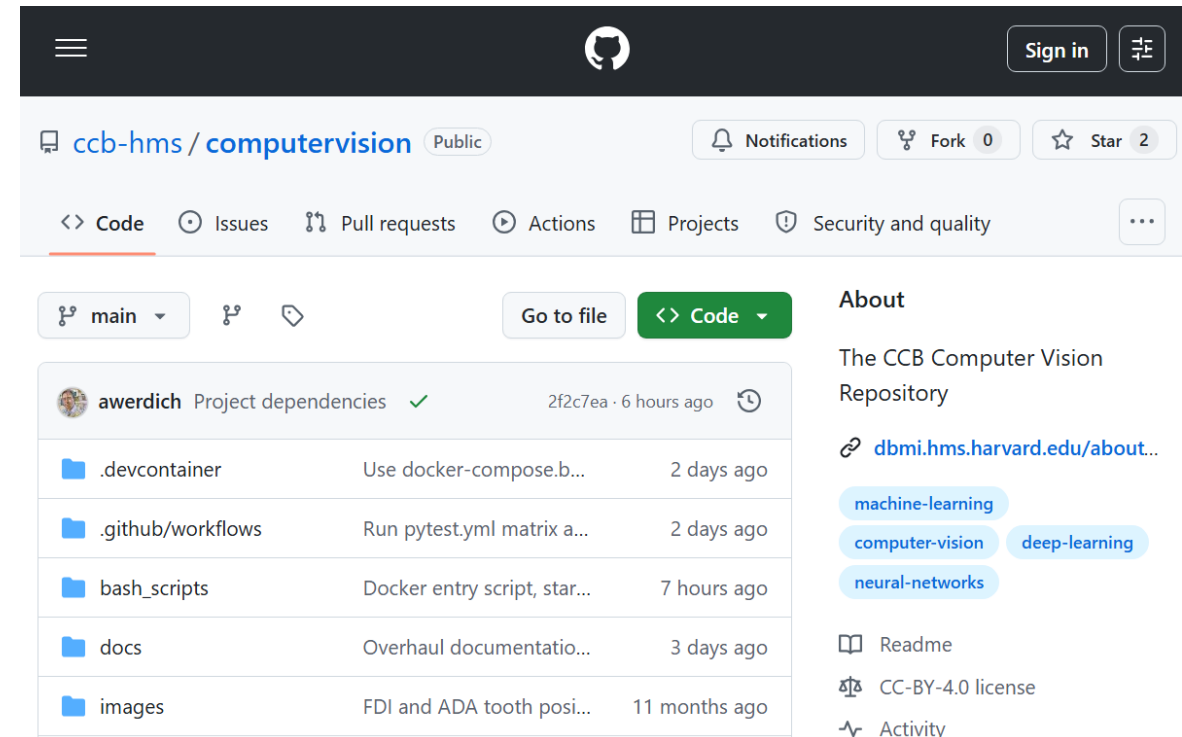
The HMS-CCB Computer Vision Repository



HARVARD
MEDICAL SCHOOL

BLAVATNIK INSTITUTE
BIOMEDICAL INFORMATICS

- Free resource for computer vision projects
- Hand-written code for
 - Image classification
 - Object detection
 - Instance segmentation
- Implementations that are hard to find
 - Grad-CAM XAI visualizations
 - mAP metric for DETR
- Feel free to use the code for your own project
- I appreciate any feedback
- Let me know if you need help



Website: <https://dbmi.hms.harvard.edu/>

Email: andreas_werdich@hms.harvard.edu

Acknowledgements



Nathan Palmer, PhD
Ludwig Geistlinger, PhD
Nidia Bucarelli, PhD
Gerald M Kiwanuka

Thank you!



Magda Feres, DDS, DMSc
Balazs Feher, DMD
Chia-Yu (Jennifer) Chen, DDS, DMSc
Caroline V Busch, DMD
Shu Tu, DMD